PLATFORM BUILT IN

lifepal

PRESENTED TO

ABOUND

PLATFORM · DATA ENGINEER

DATA ENGINEERING · FIRESIDE CHAT · NEXUS BEST PRACTICES FROM LIFEPAL

Nexus

Every change in Aurora, in the lake exactly once, proven against the source

Four years owning the data lake and pipelines behind risk and fraud models at Lifepal, a regulated insurance marketplace: CDC from PostgreSQL and MySQL into an S3 lake with AWS DMS and Debezium, Spark and Glue jobs that merge changes idempotently, and reporting, ad-hoc analytics and model training moved off the production database. This pack applies those practices to Abound's stack, Aurora → DMS → Glue → Iceberg → Athena, in a runnable repository with synthetic data. Every run figure here comes from that run; every Lifepal figure is from my CV.

PRESENTED BY

R. Firman Insan Muhammad (Vim)
Data Engineer · London

TO

Platform team
Abound

ROLE

Data Engineer
London hybrid

STACK

AWS DMS · Glue · Spark
Iceberg · Athena · Aurora

CONTENTS

What I'd like to cover

Fifteen minutes on what I've built, then the parts of the role I'd own first. Every page points to working code.

01	**Measured outcomes**	What changed at Lifepal, in numbers
02	**How my experience maps to the role**	Line by line against the job description
03	**The path**	Capture, land, apply, prove, serve
04	**Six CDC problems**	What DMS really delivers, and what each takes
05	**Proving the lake against Aurora**	The reconciliation that caught a DMS gap
06	**Table design for speed and cost**	Partitioning, sort order, compaction; Athena bytes
07	**The bridge to Data Science**	Calibration on the lake, point in time
08	**Governance and erasure**	Quality rules, tombstones, snapshot expiry
09	**On AWS**	DMS, Glue 5.0, Iceberg, Lambda and ECS on Fargate
10	**My first 90 days**	Moving workloads into the Data Lake

What changed at Lifepal

Figures from my CV and production work, described at the level I can share publicly.

99.7%
reporting latency cut: 24 h → under 5 min with CDC

75%
nightly batch cut: 6 h → 90 min by redesigning the layout

$11k
monthly platform cost against a $19k projection (≈42% under)

−60%
ad-hoc requests after self-serve datasets on the lake

−60%
data-trust escalations, with 500+ quality checks in CI

How: DMS and Kafka + Debezium from PostgreSQL and MySQL into S3; Spark and Glue jobs merging idempotently; partition pruning, Z-ordering and compaction; feature pipelines for LightGBM and XGBoost risk and fraud models.

Team: I mentor 3–5 engineers; two were promoted to Senior. Blameless post-mortems that end in durable fixes.

How my experience maps to the role

The job description's asks, what I've done, and where it shows in this repository.

THE ROLE ASKS FOR	WHAT I'VE DONE	HERE
Own the Data Lake and its ETL	Owned the S3 lake and pipelines end to end at Lifepal	src/nexus
CDC, ideally AWS DMS	DMS and Debezium from PostgreSQL and MySQL, merged idempotently	bronze.py · silver.py
Spark and Glue; RDS workloads into pipelines	Moved reporting, analytics and training off RDS	glue/cdc_job.py
Strong SQL on Aurora PostgreSQL / MySQL	Query tuning and indexing on transactional RDS	checks.py
Data structures and clustering	Partitioning, Z-order, compaction: 6 h → 90 min	layout.py
Lower AWS cost	$11k a month against a $19k projection	layout.py
Bridge to Data Science	Point-in-time training sets, parity, drift, recalibration	gold.py
Governance and standards	500+ checks in CI, PII masking, RBAC, lineage	contracts/
SOLID in practice	Connectors behind common interfaces, config injected, PyTest	src/nexus · tests/
Iceberg	Delta in production; Iceberg in this repo, one to one	silver.py · layout.py

Aurora to the lake, proven

Five steps from a commit in the lending database to a table a data scientist can trust. The run: a full load, then three months of CDC in 6 batches by arrival time.

01 · CAPTURE
DMS
Full load, then CDC: Op, cdc_ts and AR_H_CHANGE_SEQ on every row.

→

02 · LAND
Bronze
Every change keyed on (key, change_seq). A full replay changed 0 rows.

→

03 · APPLY
MERGE
Newest change_seq wins; deletes are tombstones; a reload is authoritative.

→

04 · PROVE
18/18
Row by row against Aurora; quality rules; freshness; erasure.

→

05 · SERVE
Iceberg
Partitioned and sorted for Athena; point in time for calibration.

THE RUN **59,754** source changes **417** DMS files **57,025** changes re-read on replay, **0** new **16** rows caught by reconciliation **195s** end to end

What DMS really delivers

Each is planted in the synthetic data, handled by the pipeline and covered by a test.

PROBLEM	IN THE RUN	WHAT HANDLES IT
Re-sent after a task restart	14 changes re-sent in a new file	Bronze keys on (key, change_seq): 14 skipped
A file delivered late	customers file for 2026-09-03 landed 2026-09-18	List by arrival; order by change_seq
A delete, then an older update	C00016 erased; an earlier update arrived after	Tombstone: 0 live rows after
A column added mid-stream	ALTER TABLE loans ADD COLUMN apr_bps	ADD COLUMNS in bronze, metadata only
Two changes, one timestamp	53 late payments, two statements per transaction	AR_H_CHANGE_SEQ orders them
Changes DMS never wrote	16 repayment changes, 20 Sep 02:00–04:00	Reconciliation to source, then a reload

A check failed. The run fixed it.

No key can catch a row that never arrived. Only a comparison with the source can.

16 repayments rows didn't match Aurora

DMS delivered no repayments changes between 03:00 and 04:00 on 2026-09-20, the hour of the daily direct-debit run. The source has them, so the task resumed from the wrong position after an interruption: the rows are missing, not late.

Bronze had **0** changes in that hour, against a median of **10** on other days.

Fix: Reload repayments from the source (DMS 'reload table'), apply it as authoritative, re-run the reconciliation. Then alert on hourly change volume per table so a gap pages before a month-end report finds it.

SRC/NEXUS/SILVER.PY — A RELOAD IS AUTHORITATIVE

```
MERGE INTO lake.silver.repayments t
USING silver_batch s ON t.repayment_id = s.repayment_id
WHEN MATCHED AND s.change_seq > t.change_seq
  THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE AND NOT t.is_deleted
  THEN UPDATE SET t.is_deleted = true
```

After the reload: **18/18** checks pass, including reconciliation on every table.

Read only what you need

Athena bills bytes scanned at $5 per TB. The same queries, planned from Iceberg's own file statistics on three layouts.

QUERY	NAIVE (UNPARTITIONED)	MONTHS(BOOKED_AT)	+ SORTED, COMPACTED	LESS SCANNED
Data Science: one customer's last 90 days	578 KB · 72/221 files	254 KB · 5/18	55 KB · 3/70	90%
Reporting: every transaction in September	237 KB · 28/221 files	52 KB · 2/18	54 KB · 3/70	77%

Hidden partitioning

months(booked_at): queries filter on the column, and the spec can evolve without a rewrite.

Sort order

WRITE ORDERED BY customer_id, booked_at: min/max bounds prune files for per-customer reads.

Maintenance

rewrite_data_files after MERGE; expire_snapshots and remove_orphan_files keep storage honest.

At Lifepal the same ideas, on Delta (partition pruning, Z-ordering, compaction, AQE), cut the nightly batch from 6 h to 90 min.

Calibrate on the lake, as of the decision

Calibration needs the score at decision time, what happened next, and every feature as it was known then.

SCORE BAND	LOANS	PREDICTED	OBSERVED
0–10%	59	7%	8%
10–20%	164	15%	19%
20%+	98	25%	30%

gold.ds_calibration_by_band: the scorecard is too optimistic above 10%, and that gap is what recalibration fixes.

Point in time, or a leak

Open Banking enrichment relabels salary credits days after the fetch. A feature built from today's labels differs for **72 of 111** decisions, adding **£2,807** of income on average that the model never saw.

At Lifepal: feature pipelines behind the customer-risk and claims-fraud models: point-in-time training sets, training/serving parity, drift monitoring and recalibration.

Standards that run as code

Quality rules from the contract, reconciliation, freshness and erasure: checked on every run, failing loudly.

Contracts

contracts/sources.yml: keys, types, PII columns, what to reconcile. quality.yml: the rules on silver.

Erasure

Deletes become tombstones with values nulled; bronze is purged; expire_snapshots on 10 tables so time travel can't recover PII.

Freshness

Measured on a table that changes every day; in production, the DMS heartbeat and CDCLatencySource.

At Lifepal: 500+ quality checks in CI, freshness and volume monitoring with alerting, PII masking, role-based access and lineage. Data-trust escalations fell by 60%.

09 · ON AWS

The same code, deployed

Written to be read and reviewed; not applied, since there's no AWS account behind a portfolio.

CAPTURE

infra/dms/

S3 target: CSV with header, TimestampColumnName=cdc_ts, AR_H_CHANGE_SEQ → change_seq.

APPLY

glue/cdc_job.py

Glue 5.0 (Spark 3.5, Iceberg 1.7) with the Glue Data Catalog; the same bronze and silver code.

AROUND IT

Lambda · ECS on Fargate

Lambda on S3 events and failed checks; Fargate tasks for reloads, backfills and the reconciliation export.

PROVISION

infra/terraform/

KMS, S3, DMS instance, endpoints and task, Glue job and schedule, least-privilege IAM.

Moving workloads into the Data Lake

The role's first 6–12 months, as I'd start them.

Days 1–30 · Listen and ship

Learn the lake, the DMS tasks and the models from their owners. A small fix in production early.

Days 31–60 · Prove it

Row-by-row reconciliation from Aurora for the tables Data Science uses; a change-volume alert per table.

Days 61–90 · Move one workload

Model calibration on the lake, point in time, with Data Science; the RDS query switched off.

Day 90 · Measured

Fresher data, a smaller Athena bill per query, and one fewer heavy read on Aurora.

PROVEN AT LIFEPAL **24h → <5m** reporting latency · **6h → 90m** nightly batch · **$11k** vs $19k projection · **−60%** ad-hoc requests · **500+** checks in CI · **−60%** data-trust escalations

Everything in this pack runs at **rfim.github.io/nexus-abound**. Synthetic data; an independent application, not affiliated with or endorsed by Abound.